



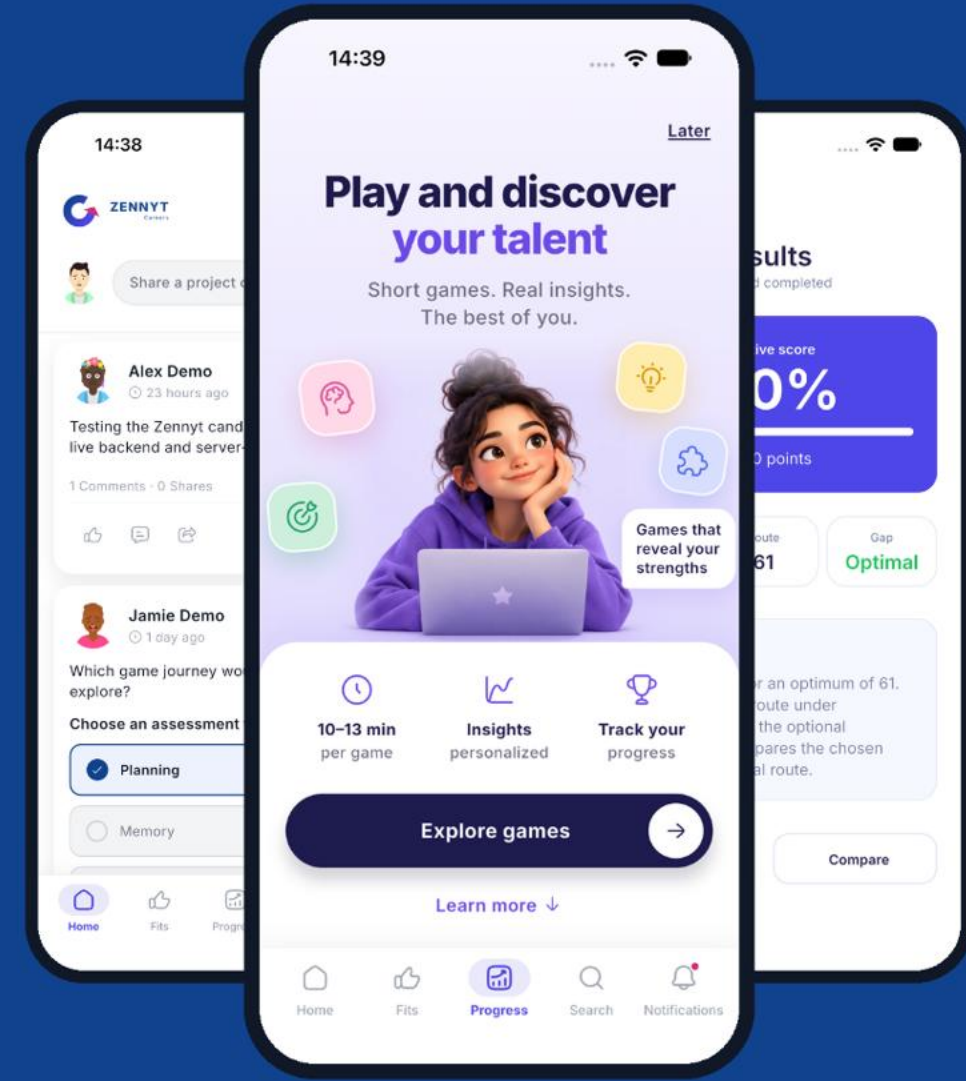
Summer internship defense · 2026

Building the cognitive games of the Zennyt career platform

Mohamed Slimane · Full Stack Mobile Developer

1 June – 30 August 2026

Supervisors: Mehdi Mtir, Abdelmonem Aisa



The internship at a glance



3

months

1 June – 30 August 2026



14

mini-games

across 9 cognitive families



6

languages

EN · FR · ES · DE · IT · NL



2,372

tests passing

843 backend · 1,529 Flutter

Today

1

Context & objectives

2

Architecture & stack

3

**My contribution:
the games**

4

Testing & results

5

Lessons & next steps

One platform, three audiences

Zennyt is a mobile career platform where profiles, job offers, assessments and conversations meet.



Candidates

Build a profile, explore offers, play cognitive games, take tests and chat with recruiters.



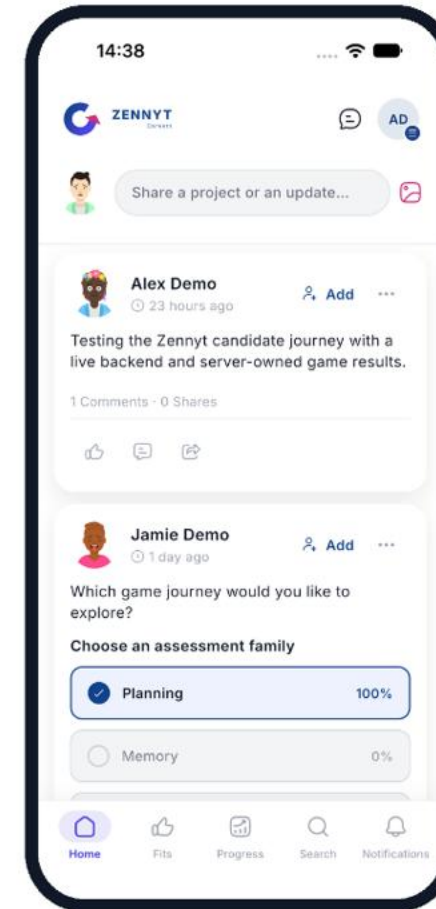
Recruiters

Publish offers, author technical tests and review applicants with a Fit Score.

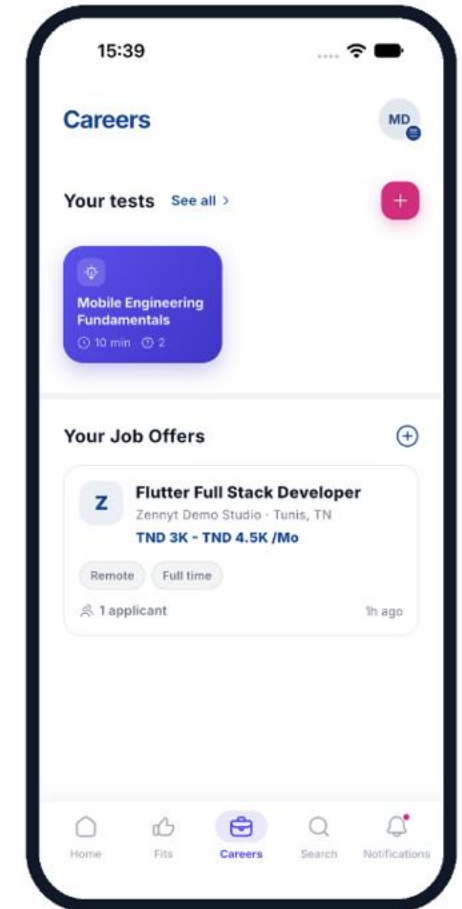


Staff

Game Studio and Platform Console manage assessment content and operations.



Candidate · Home feed



Recruiter · Careers dashboard

Challenge: trustworthy results



Why it is hard



The phone captures every action and timing precisely



The backend validates the session and applies the scoring rule



The result stays consistent across games, languages and screens

Objectives

01

Implement interactive Flutter game journeys

02

Connect raw measurements to server-side scoring

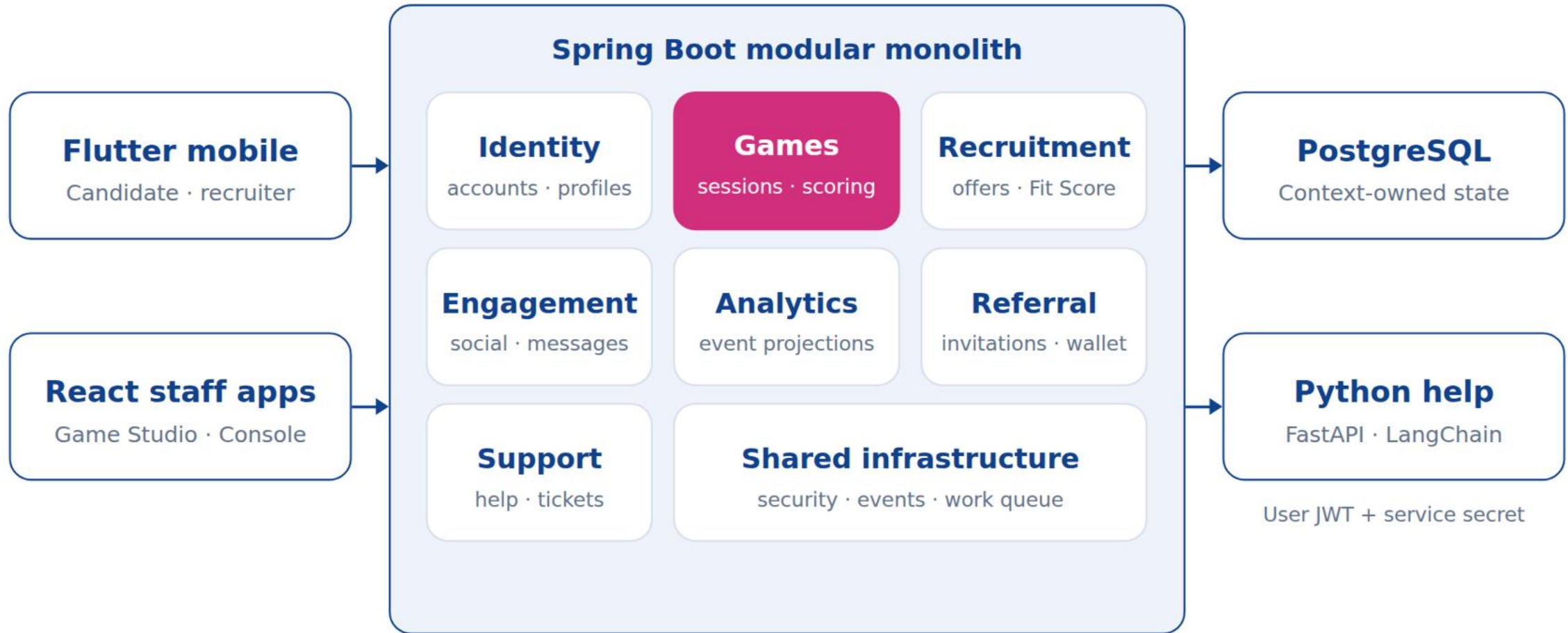
03

Manage sessions, tutorials and lifecycle states

04

Reusable result screens in six languages

A modular monolith behind two clients



Each context: API → application → pure domain → infrastructure, checked by ArchUnit. Contexts talk through domain events.

Inside the Games context



Games

sessions · scoring — the context I worked in

GameSession

Player, family, status, attempts and timestamps

MiniGame

The tasks that belong to each family

Score

Raw, maximum and normalized values

Snapshot + form

Published settings frozen at start; assigned scenario content

A session completes only when every required mini-game has a valid recorded attempt.

Technology stack by layer



Mobile



Riverpod state
go_router
Flame game runtime
Dio + STOMP
Secure storage, Hive

Backend



Spring Boot monolith
Spring Data JPA
Spring Security,
OAuth2
OpenAPI Generator

Administration



TanStack Start
Game Studio
Platform Console

Help service



LangChain
Chroma retrieval
Authenticated
gateway

Quality & runtime JUnit · ArchUnit · Flutter test · OpenAPI contract-parity tests · Compose · Maven

My focus: 9 families, 14 mini-games



Planifik

Optimal Path · Task
Scheduling · Predictive
Puzzle



Move Fast

Move Fast Core



Memory Quest

I Investigate



Decision

I Decide



Emotional Regulation

Emotional Radar · Reflective
Pause · Strategic Choices



Continuous Attention

I Continue



Visuomotor Coordination

I Coordinate



Visuospatial Memory

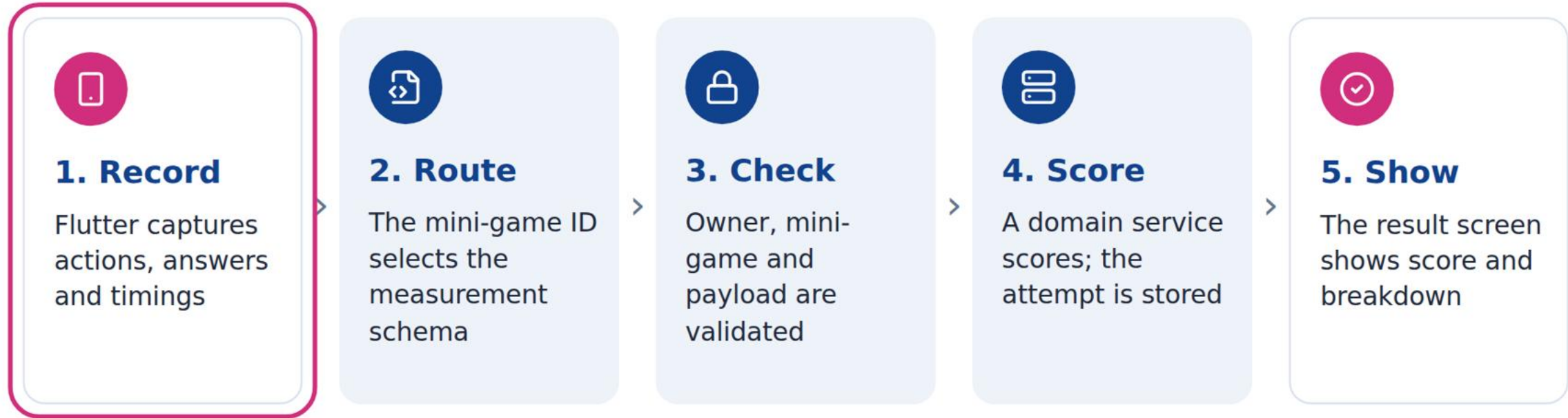
I Place



Decision Behavioral

BART · IST

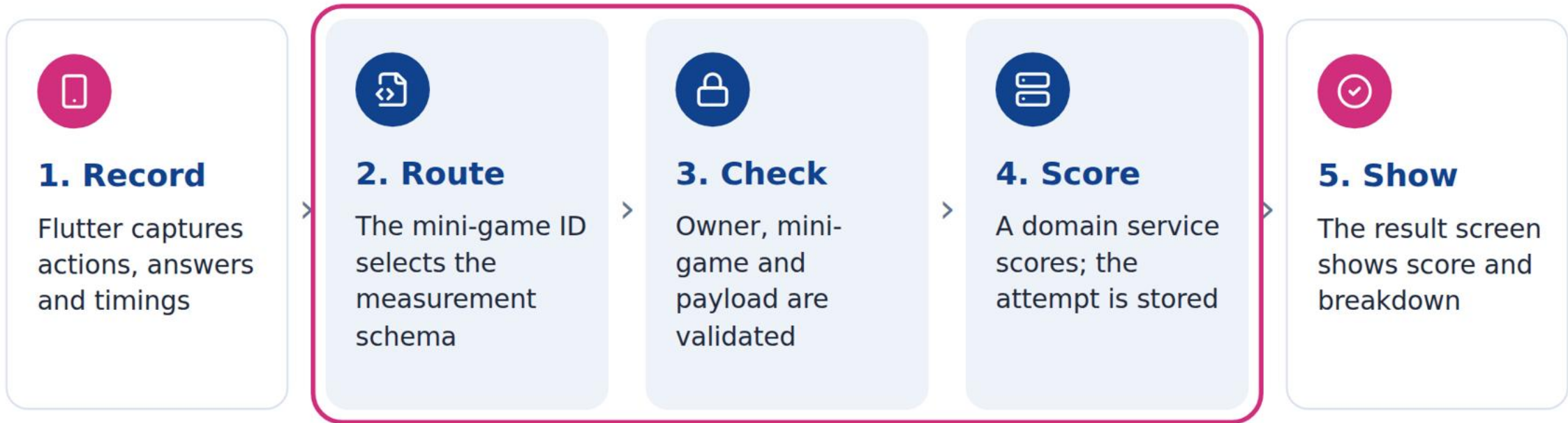
Client measures, server decides



ON THE PHONE

Flutter records raw observations: what was tapped, answered and when. Never a score.

Client measures, server decides



ON THE SERVER

The schema is picked by mini-game ID, the session is checked, and a domain service computes the score.

Client measures, server decides



1. Record

Flutter captures actions, answers and timings



2. Route

The mini-game ID selects the measurement schema



3. Check

Owner, mini-game and payload are validated



4. Score

A domain service scores; the attempt is stored



5. Show

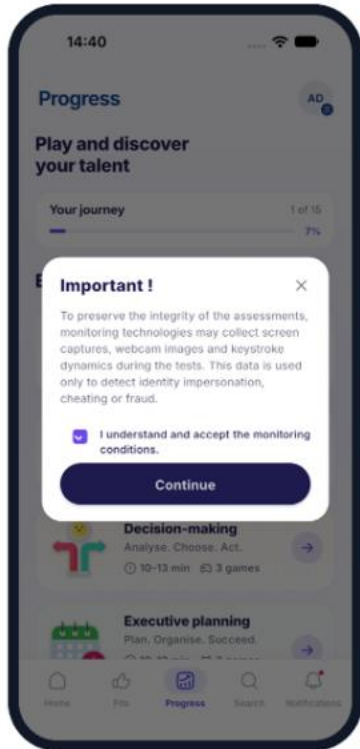
The result screen shows score and breakdown



The client never chooses an authoritative score

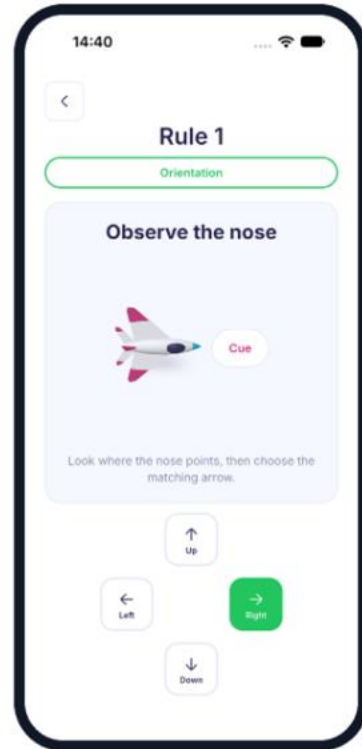
Invalid metrics, foreign sessions, incompatible mini-games and duplicates are rejected. Invalid runs are kept for audit, never counted.

One shared journey for every game



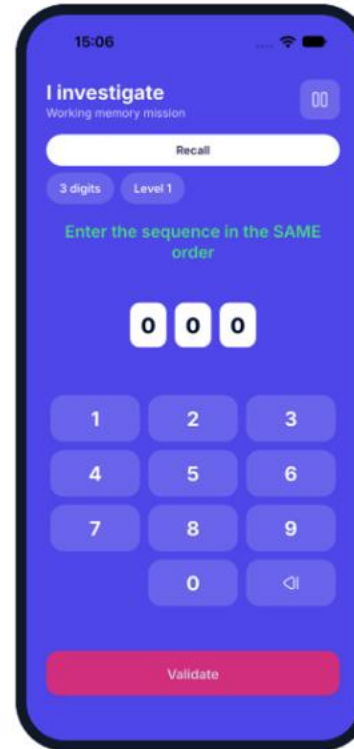
1 Consent

Monitoring conditions



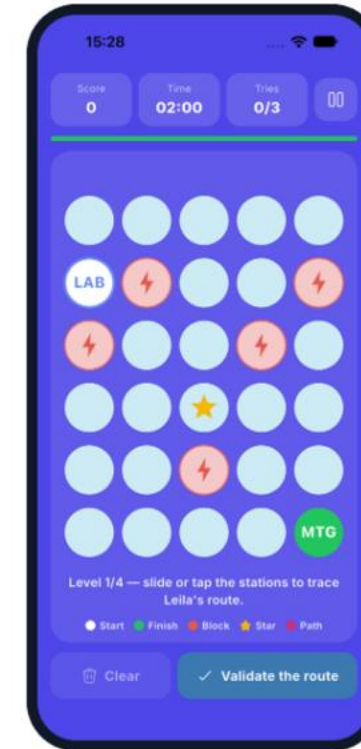
2 Instructions

Move Fast rule



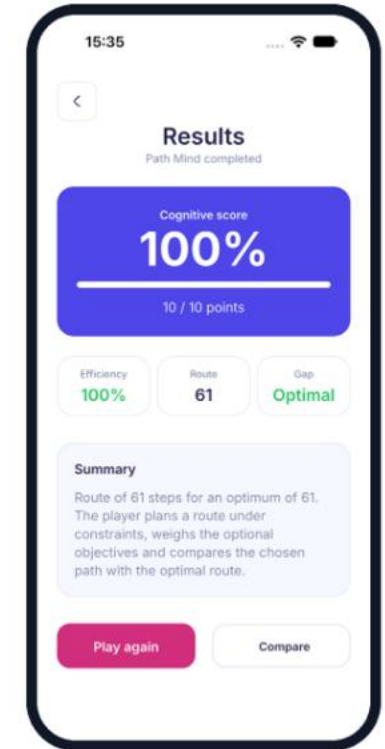
3 Gameplay

Memory Quest recall



4 Planning

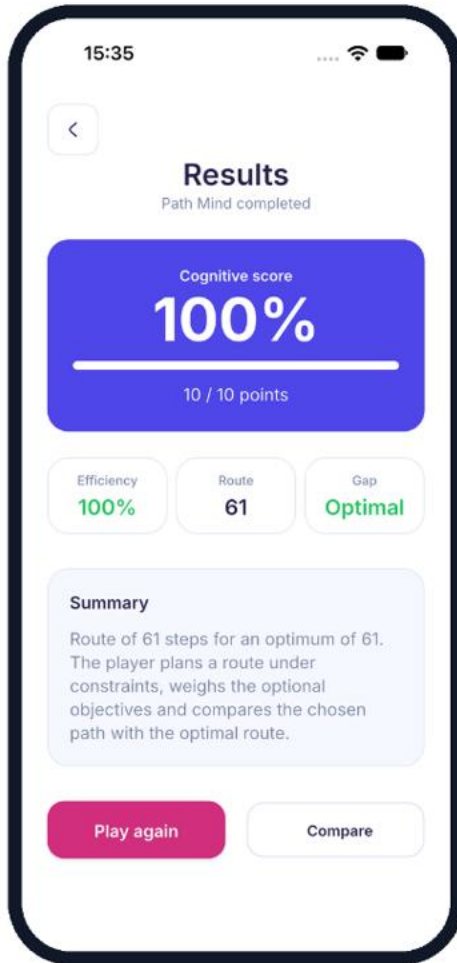
Optimal Path board



5 Results

Server score: 10 / 10

Results come back from the server



100%

Cognitive score: 10 of 10 points, computed by the backend

61 / 61

Route steps against the server-derived optimum

3

Separate layers: raw performance, normalized score, indicators

Shared result screens: score, breakdown, explanation, then Play again or Compare.

Replay: the server rebuilds the run



BART

Pumps and cash-outs per balloon → explosions and earnings rebuilt on the server



IST

Opened boxes and choices → what was visible at decision time

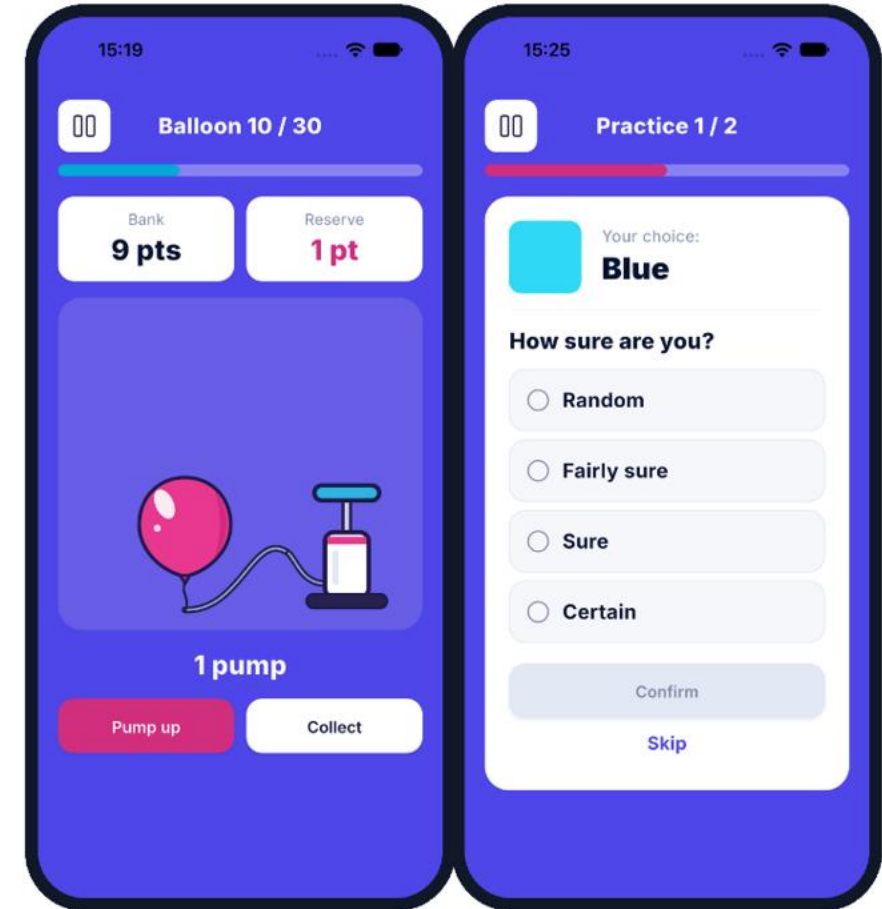


I Place

Placement actions replayed against the controlled layout

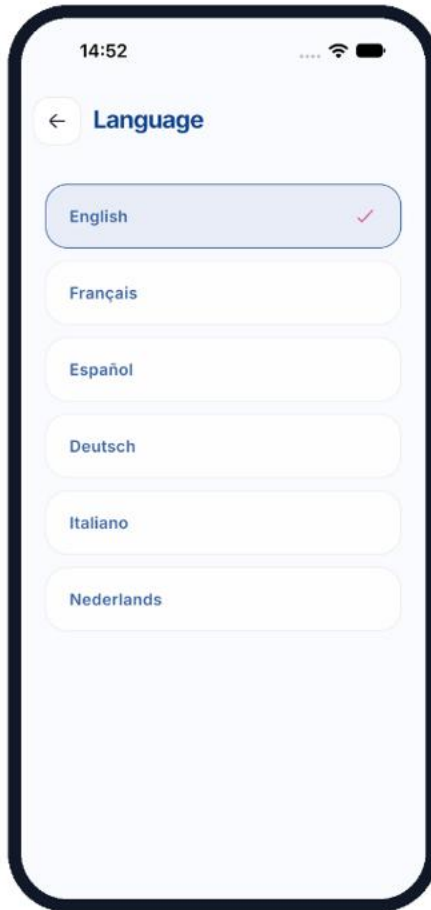
Controlled content

Runtime snapshots freeze settings at session start; assigned forms bind scenario games to a bank.

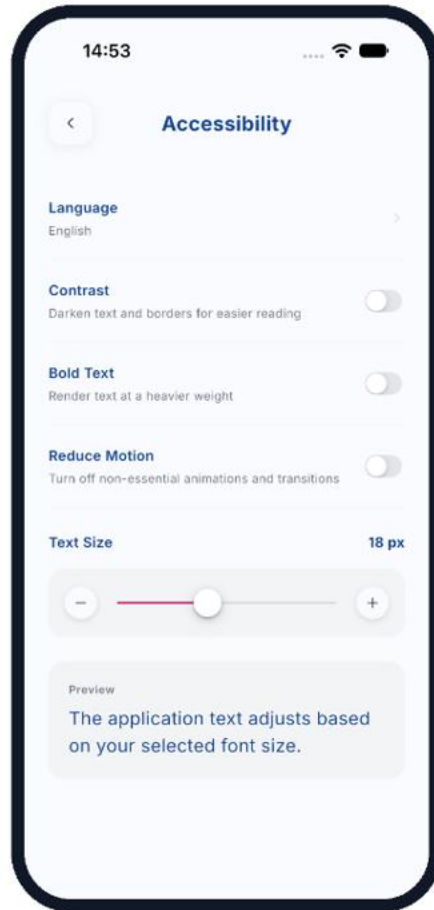


BART · balloon 10 of 30 IST · choice + confidence

Six languages, adjustable reading



Language picker



Accessibility settings

English

Français

Español

Deutsch

Italiano

Nederlands



Localization keys everywhere

Instructions, controls, errors, accessibility labels; ICU plurals.



Layouts that absorb long strings

German and Dutch stay readable on small screens.



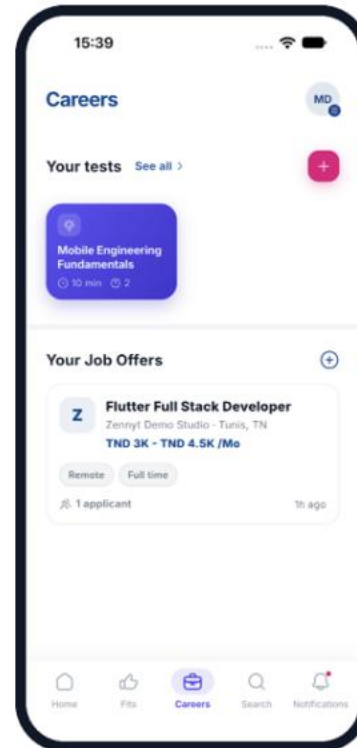
Reading preferences

Text size, contrast, bold text and reduced motion.

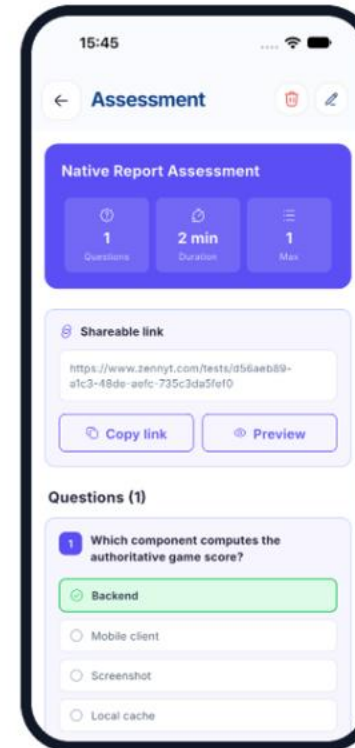
From recruiter test to application

Beyond the games, I tested the full path on a real iPhone build.

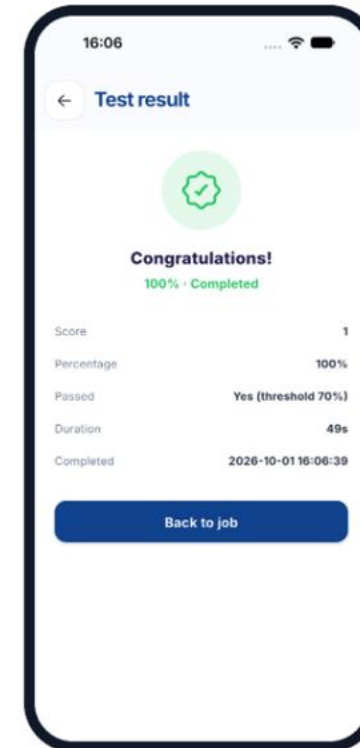
- 1 **Recruiter** creates a test and publishes an offer
- 2 **Candidate** accepts consent, answers shuffled questions
- 3 **Server** grades the attempt: 100%
- 4 **Candidate** applies with the passed-test banner



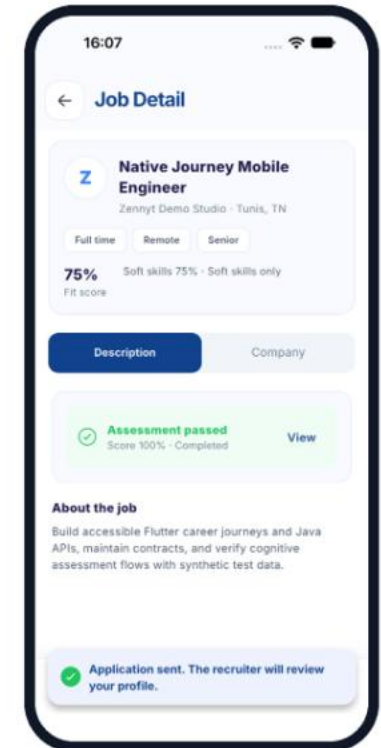
Careers dashboard



Test and answers



Server result: 100%



Application sent

Automated suites are green



843

backend Maven tests passed

0 failed · 41 skipped

1,529

Flutter tests passed

0 failed · 0 skipped

218 / 221

API checks passed

2 failed · 1 blocked



Test environment

Spring Boot dev backend, local PostgreSQL, Flutter iPhone app driven by XCTest, synthetic data.



Not yet covered

Email/OTP, push, call media, real payments, multi-device. AI help gateway returned 503.

Walkthrough: 1 passed, 6 partial



Use case	Workflow	Status	Principal observation
UC-01	Cognitive assessment	Partial	Planning result saved; memory payload failed
UC-02	Offer management	Partial	Offer published; test picker failed
UC-03	Technical assessment	Passed	Completed; result read back by both parties
UC-04	Communication	Partial	Message persisted; call media unverified
UC-05	Community	Partial	Post and comment persisted; self-friend action
UC-06	Preferences and help	Partial	Preferences persisted; AI service down
UC-07	Referrals and plans	Partial	Local state verified; no real payouts

Partial: the core path persisted, but a dependent step or external service failed or was not verified.

Five defects found and documented



Area	What happened	Fix direction
 Memory Quest	Client sends mode, not memoryQuestMode: result not saved	Align payload with contract
 Decision forms	Another candidate could read a player's form	Ownership check on read
 Assessment picker	/assessments/pick parsed as an ID → HTTP 400	Static route before /{id}
 Localization	French notifications during an English session	Use the session locale
 Community	Own post offered to add the author as a friend	Hide actions on own posts

What I take away



“Treat a game as a complete feature: instructions, interaction, measurement, validation, scoring and feedback must stay consistent from Flutter to the backend.”

Skills developed

Mobile ↔ backend integration

Game protocols as typed payloads

Async state across screens

Next steps

- 1 Fix the five defects, starting with the ownership check
- 2 Integration-test email/OTP, push, calls and payments
- 3 Bring the AI help service online and re-run the walkthrough



Thank you

Questions and discussion

Mohamed Slimane · Full Stack Mobile Developer · Zennyt